

Design And Development Of Data Storage Solutions For Analysis



Group Members:

Adwaitha Shyam (20068181),

Anantha Krishnan Kunnappilly (20070043),

Aswin Vaduvana Babu (20063083)

Module: Data Storage Solutions for Data Analytics (B9DA111)

1. INTRODUCTION

This report presents the design and implementation of a Decision Support System (DSS) developed using data warehousing and business intelligence techniques. The system is based on a hotel booking dataset and demonstrates the complete lifecycle of a DSS, including data warehouse design, Extract–Transform–Load (ETL) processes, reporting, visualisation, and a comparison between relational and graph databases.

The project was implemented using Microsoft SQL Server for data warehousing, SQL-based ETL, SQL Server Reporting Services (SSRS) for reporting, Tableau for advanced visualisation, and Neo4j for graph database implementation. The objective is to demonstrate how structured and unstructured data models can support analytical decision-making.

2. SOURCE OF DATA

The dataset used in this project is the **Hotel Booking Demand Dataset**, obtained from Kaggle. The dataset contains booking information for both city hotels and resort hotels. It includes attributes related to customer demographics, booking dates, cancellations, market segments, room types, and pricing information.

The dataset consists of approximately **119,000 records**, representing real-world transactional data. It is suitable for decision support analysis as it allows the exploration of revenue trends, customer behaviour, cancellation patterns, and hotel performance over time.

Data Format: CSV

Nature of Data: Historical transactional data

Use Case: Hotel management decision support

3. VISION AND BUSINESS REQUIREMENTS FOR THE DATA WAREHOUSE

Vision:

The vision of the data warehouse is to provide hotel management with a centralised, integrated, and historical data repository that supports strategic and tactical decision-making. The system enables analysis of hotel performance, customer behaviour, revenue trends, and operational efficiency.

Business Requirements:

The following business requirements were identified:

- Analyse total and monthly revenue trends

- Compare performance between city and resort hotels
- Identify cancellation patterns across market segments
- Understand customer demographics and booking behaviour
- Support management reporting and forecasting
- Enable interactive and parameterised reporting

4. DATA WAREHOUSE DESIGN

A **star schema** design was implemented to support efficient analytical querying. The design consists of a central fact table connected to multiple dimension tables.

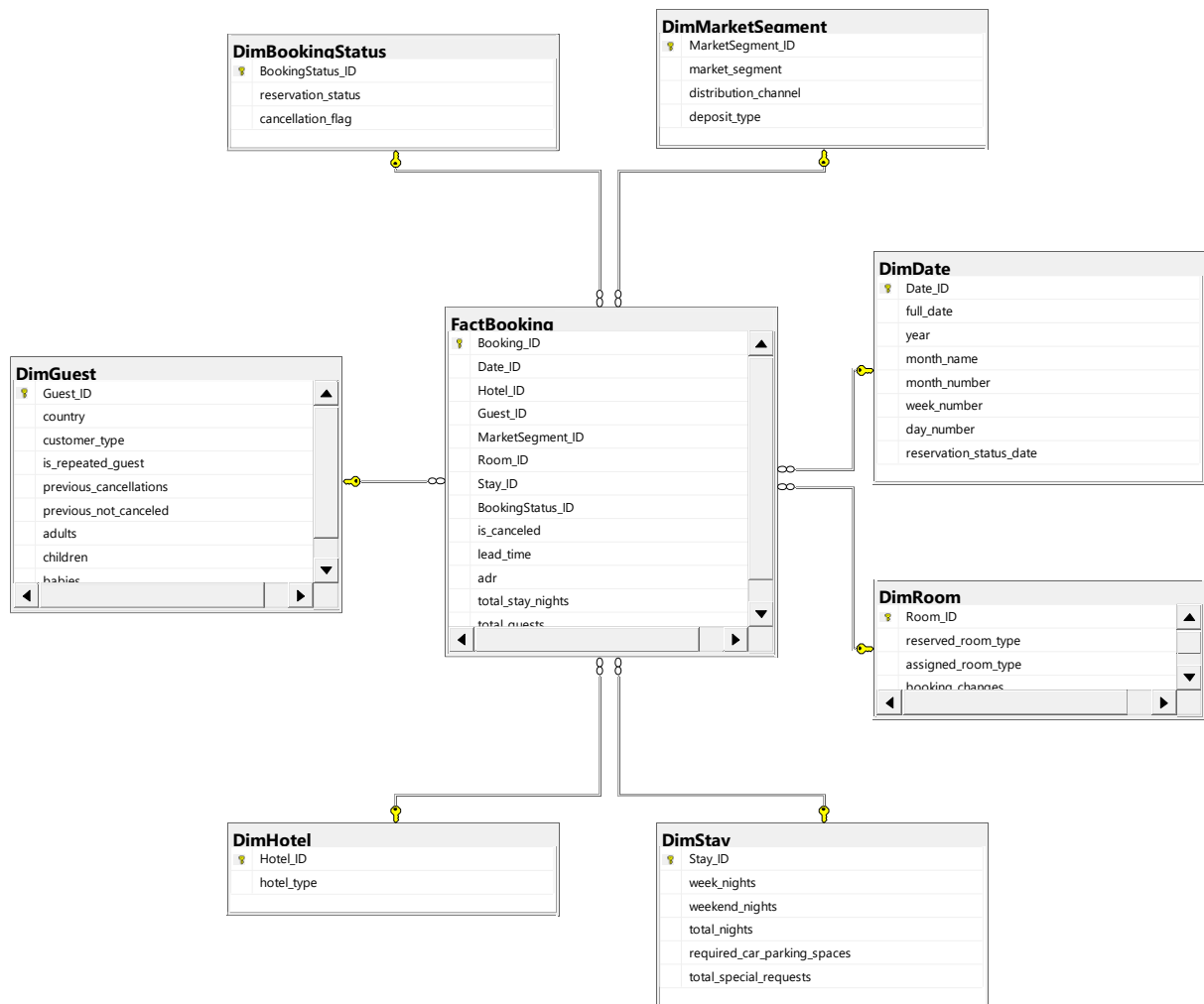
Fact Table:

- **FactBooking**
 - Measures include: revenue (ADR), total stay nights, number of guests, lead time, and cancellation status.

Dimension Tables:

- **DimDate** – stores date, month, and year attributes
- **DimHotel** – stores hotel type (city or resort)
- **DimGuest** – stores customer demographic details
- **DimMarketSegment** – stores market and distribution channel information
- **DimRoom** – stores room allocation details
- **DimStay** – stores length of stay information
- **DimBookingStatus** – stores reservation and cancellation status

This design minimises redundancy and improves query performance for analytical workloads.



5. ETL Implementation

Overview:

ETL was implemented using SQL statements in Microsoft SQL Server. The operational booking data was first imported into a staging table (dbo.hotel_bookings) from the CSV file. From this staging table, data was extracted, transformed, and then loaded into the star schema data warehouse (HotelDW) consisting of multiple dimension tables and a central fact table (FactBooking).

Extraction:

The extraction step used the staging table as the operational source. The following checks were performed to confirm successful extraction:

- A staging row count check to confirm the number of rows imported:
- `SELECT COUNT(*) AS StagingRows FROM dbo.hotel_bookings;`
- A sample extraction query (TOP 20) was used to inspect important attributes and verify calculated fields before loading.

```
USE HotelDW;
SELECT COUNT(*) AS StagingRows FROM dbo.hotel_bookings;

SELECT TOP 20
    hotel,
    arrival_date_year,
    arrival_date_month,
    arrival_date_day_of_month,
    DATEFROMPARTS(
        arrival_date_year,
        CASE LTRIM(RTRIM(arrival_date_month))
            WHEN 'January' THEN 1 WHEN 'February' THEN 2
            WHEN 'March' THEN 3 WHEN 'April' THEN 4
            WHEN 'May' THEN 5 WHEN 'June' THEN 6
            WHEN 'July' THEN 7 WHEN 'August' THEN 8
            WHEN 'September' THEN 9 WHEN 'October' THEN 10
            WHEN 'November' THEN 11 WHEN 'December' THEN 12
        END,
        arrival_date_day_of_month
    ) AS full_date_calc,
    (ISNULL(stays_in_week_nights,0) + ISNULL(stays_in_weekend_nights,0)) AS total_stay_nights_calc,
    (ISNULL(adults,0) + ISNULL(children,0) + ISNULL(babies,0)) AS total_guests_calc,
    adr
FROM dbo.hotel_bookings;

USE HotelDW;
GO
```

Transformation:

Transformations were performed in SQL during the dimension and fact loading processes. Key transformation methods used include:

1. Date standardisation

- The dataset contains arrival_date_year, arrival_date_month (as text), and arrival_date_day_of_month.
- A full date was derived using DATEFROMPARTS(...), converting month names into month numbers using a CASE expression combined with LTRIM/RTRIM to handle spacing issues.
- This produced a consistent full_date_calc, which was loaded into DimDate.full_date.

2. Derived metrics

Two main derived measures were computed:

- Total nights

total_nights_calc = stays_in_week_nights + stays_in_weekend_nights

- Total guests

total_guests_calc = adults + children + babies

These derived values were later stored in the Fact table fields total_stay_nights and total_guests.

3. Null handling and data quality

- Null values in numeric fields (e.g., children, babies, night stays) were handled using ISNULL(..., 0) to prevent load failures and ensure consistent aggregations.
- Missing country values were standardised using ISNULL(country, '') to avoid joining issues and to allow a consistent Guest dimension record.

4. Data type conversions

- Numeric fields imported from CSV can arrive as strings. Conversions were applied using CAST / TRY_CAST to ensure compatibility with warehouse column types.
- Examples include converting is_repeated_guest, previous_cancellations, lead_time, and night counts to integers, and adr to a numeric type suitable for analysis.

```
35
36      -- 2) Then delete from Dimensions
37      DELETE FROM dbo.DimBookingStatus;
38      DELETE FROM dbo.DimStay;
39      DELETE FROM dbo.DimRoom;
40      DELETE FROM dbo.DimMarketSegment;
41      DELETE FROM dbo.DimGuest;
42      DELETE FROM dbo.DimHotel;
43      DELETE FROM dbo.DimDate;
44      GO
45
46
47
48
49      DBCC CHECKIDENT ('dbo.FactBooking', RESEED, 0);
50      DBCC CHECKIDENT ('dbo.DimBookingStatus', RESEED, 0);
51      DBCC CHECKIDENT ('dbo.DimStay', RESEED, 0);
52      DBCC CHECKIDENT ('dbo.DimRoom', RESEED, 0);
53      DBCC CHECKIDENT ('dbo.DimMarketSegment', RESEED, 0);
54      DBCC CHECKIDENT ('dbo.DimGuest', RESEED, 0);
55      DBCC CHECKIDENT ('dbo.DimHotel', RESEED, 0);
56      DBCC CHECKIDENT ('dbo.DimDate', RESEED, 0);
57
58
59
60      INSERT INTO dbo.DimHotel (hotel_type)
61      SELECT DISTINCT hotel
62      FROM dbo.hotel_bookings;
63
```

00 % No issues found

Loading:

The loading process followed standard data warehousing practice: dimensions first, then the fact table.

1) Dimension Loads

Each dimension was loaded using INSERT INTO ... SELECT DISTINCT ... FROM dbo.hotel_bookings to ensure the dimension tables contained unique business attribute combinations, while identity columns in dimensions acted as surrogate keys.

Loaded dimensions included:

- DimHotel (hotel type)
- DimMarketSegment (market segment, distribution channel, deposit type)
- DimRoom (room types and booking changes)
- DimStay (weekday/weekend nights and derived totals)
- DimBookingStatus (reservation status and cancellation flag)
- DimGuest (guest attributes and booking history)
- DimDate (derived full date + date attributes)

2) Reload Strategy (Reset + Reseed)

To support repeatable ETL runs during development and testing:

- Data was cleared from the warehouse tables (Fact first, then dimensions).
- Identity keys were reset using DBCC CHECKIDENT(..., RESEED, 0) to restart surrogate key numbering consistently on each reload.

-----Reloading Dimensions-----

```
INSERT INTO dbo.DimMarketSegment (market_segment, distribution_channel, deposit_type)
SELECT DISTINCT market_segment, distribution_channel, deposit_type
FROM dbo.hotel_bookings;
```

```
INSERT INTO dbo.DimRoom (reserved_room_type, assigned_room_type, booking_changes)
SELECT DISTINCT reserved_room_type, assigned_room_type, booking_changes
FROM dbo.hotel_bookings;
```

```
INSERT INTO dbo.DimStay (
    week_nights, weekend_nights, total_nights,
    required_car_parking_spaces, total_special_requests
)
SELECT DISTINCT
    stays_in_week_nights,
    stays_in_weekend_nights,
    stays_in_week_nights + stays_in_weekend_nights,
    required_car_parking_spaces,
    total_of_special_requests
FROM dbo.hotel_bookings;
```

```
INSERT INTO dbo.DimBookingStatus (reservation_status, cancellation_flag)
SELECT DISTINCT reservation_status, is_canceled
FROM dbo.hotel_bookings;
```

```
INSERT INTO dbo.DimGuest (
    country, customer_type, is_repeated_guest,
    previous_cancellations, previous_not_canceled,
```

```

94         adults, children, babies
95     )
96     SELECT DISTINCT
97         ISNULL(country, '') AS country,
98         customer_type,
99         CAST(is_repeated_guest AS INT),
100        CAST(previous_cancellations AS INT),
101        CAST(previous_bookings_not_canceled AS INT),
102        CAST(adults AS INT),
103        ISNULL(CAST(children AS INT), 0),
104        ISNULL(CAST(babies AS INT), 0)
105 FROM dbo.hotel_bookings;
106
107 INSERT INTO dbo.DimDate (
108     full_date, year, month_name, month_number,
109     week_number, day_number, reservation_status_date
110 )
111 SELECT DISTINCT
112     DATEFROMPARTS(
113         arrival_date_year,
114         CASE LTRIM(RTRIM(arrival_date_month))
115             WHEN 'January' THEN 1 WHEN 'February' THEN 2
116             WHEN 'March' THEN 3 WHEN 'April' THEN 4
117             WHEN 'May' THEN 5 WHEN 'June' THEN 6
118             WHEN 'July' THEN 7 WHEN 'August' THEN 8
119             WHEN 'September' THEN 9 WHEN 'October' THEN 10
120             WHEN 'November' THEN 11 WHEN 'December' THEN 12
121         END,
122         arrival_date_day_of_month
123     ),

```

```

123     ),
124     arrival_date_year,
125     arrival_date_month,
126     CASE LTRIM(RTRIM(arrival_date_month))
127         WHEN 'January' THEN 1 WHEN 'February' THEN 2
128         WHEN 'March' THEN 3 WHEN 'April' THEN 4
129         WHEN 'May' THEN 5 WHEN 'June' THEN 6
130         WHEN 'July' THEN 7 WHEN 'August' THEN 8
131         WHEN 'September' THEN 9 WHEN 'October' THEN 10
132         WHEN 'November' THEN 11 WHEN 'December' THEN 12
133     END,
134     arrival_date_week_number,
135     arrival_date_day_of_month,
136     reservation_status_date
137 FROM dbo.hotel_bookings;
138

```

3) Fact Load

The Fact table (FactBooking) was loaded after all dimensions were populated. A CTE (WITH hb AS (...)) was used to:

- compute full_date_calc, total_nights_calc, and total_guests_calc once,
- then insert into the Fact table by joining staging rows (hb) to each dimension to retrieve the correct surrogate keys.

The Fact load maps each booking record to:

- Date_ID
- Hotel_ID
- Guest_ID
- MarketSegment_ID
- Room_ID
- Stay_ID
- BookingStatus_ID

This ensures the Fact table is fully conformed to the star schema and supports fast analytical querying.

```
144 ;WITH hb AS (  
145     SELECT  
146         hb.*,  
147         DATEFROMPARTS(  
148             hb.arrival_date_year,  
149             CASE LTRIM(RTRIM(hb.arrival_date_month))  
150                 WHEN 'January' THEN 1 WHEN 'February' THEN 2  
151                 WHEN 'March' THEN 3 WHEN 'April' THEN 4  
152                 WHEN 'May' THEN 5 WHEN 'June' THEN 6  
153                 WHEN 'July' THEN 7 WHEN 'August' THEN 8  
154                 WHEN 'September' THEN 9 WHEN 'October' THEN 10  
155                 WHEN 'November' THEN 11 WHEN 'December' THEN 12  
156             END,  
157             hb.arrival_date_day_of_month  
158         ) AS full_date_calc,  
159         (ISNULL(hb.stays_in_week_nights,0) + ISNULL(hb.stays_in_weekend_nights,0)) AS total_nights_calc,  
160         (ISNULL(hb.adults,0) + ISNULL(hb.children,0) + ISNULL(hb.babies,0)) AS total_guests_calc  
161     FROM dbo.hotel_bookings hb  
162 )  
163 INSERT INTO dbo.FactBooking (  
164     Date_ID, Hotel_ID, Guest_ID, MarketSegment_ID, Room_ID, Stay_ID, BookingStatus_ID,  
165     is_canceled, lead_time, adr, total_stay_nights, total_guests  
166 )  
167 SELECT  
168     d.Date_ID,  
169     h.Hotel_ID,  
170     g.Guest_ID,  
171     m.MarketSegment_ID,
```

```

171         m.MarketSegment_ID,
172         r.Room_ID,
173         s.Stay_ID,
174         bs.BookingStatus_ID,
175         hb.is_canceled,
176         hb.lead_time,
177         hb.adr,
178         hb.total_nights_calc,
179         hb.total_guests_calc
180 FROM hb
181 JOIN dbo.DimHotel h
182     ON h.hotel_type = hb.hotel
183 JOIN dbo.DimMarketSegment m
184     ON m.market_segment = hb.market_segment
185     AND m.distribution_channel = hb.distribution_channel
186     AND m.deposit_type = hb.deposit_type
187 JOIN dbo.DimRoom r
188     ON r.reserved_room_type = hb.reserved_room_type
189     AND r.assigned_room_type = hb.assigned_room_type
190     AND r.booking_changes = hb.booking_changes
191 JOIN dbo.DimStay s
192     ON s.week_nights = hb.stays_in_week_nights
193     AND s.weekend_nights = hb.stays_in_weekend_nights
194     AND s.total_nights = hb.total_nights_calc
195     AND s.required_car_parking_spaces = hb.required_car_parking_spaces
196     AND s.total_special_requests = hb.total_of_special_requests
197 JOIN dbo.DimBookingStatus bs
198     ON bs.reservation_status = hb.reservation_status

```

100 % No issues found

```

199     AND bs.cancellation_flag = hb.is_canceled
200 JOIN dbo.DimGuest g
201     ON ISNULL(g.country, '') = ISNULL(hb.country, '')
202     AND g.customer_type = hb.customer_type
203     AND g.is_repeated_guest = hb.is_repeated_guest
204     AND g.previous_cancellations = hb.previous_cancellations
205     AND g.previous_not_canceled = hb.previous_bookings_not_canceled
206     AND g.adults = hb.adults
207     AND g.children = ISNULL(hb.children, 0)
208     AND g.babies = ISNULL(hb.babies, 0)
209 JOIN dbo.DimDate d
210     ON d.full_date = hb.full_date_calc
211     AND d.reservation_status_date = hb.reservation_status_date;

```

Validation:

After loading, row-count checks were performed across the Fact and all Dimension tables using a single validation query (UNION ALL). This provided evidence that:

- dimensions were populated successfully,
- the Fact table load completed,
- and the warehouse contains the expected volume of data.

Example validation output includes:

- FactBooking rows ≈ 119,390 (matching the dataset scale)

```
215  -----Validate ETL-----
216
217  SELECT 'FactBooking' AS TableName, COUNT(*) AS Rows FROM dbo.FactBooking
218  UNION ALL SELECT 'DimDate', COUNT(*) FROM dbo.DimDate
219  UNION ALL SELECT 'DimHotel', COUNT(*) FROM dbo.DimHotel
220  UNION ALL SELECT 'DimGuest', COUNT(*) FROM dbo.DimGuest
221  UNION ALL SELECT 'DimMarketSegment', COUNT(*) FROM dbo.DimMarketSegment
222  UNION ALL SELECT 'DimRoom', COUNT(*) FROM dbo.DimRoom
223  UNION ALL SELECT 'DimStay', COUNT(*) FROM dbo.DimStay
224  UNION ALL SELECT 'DimBookingStatus', COUNT(*) FROM dbo.DimBookingStatus;
```

100 % No issues found

Results Messages

	TableName	Rows
1	FactBooking	119390
2	DimDate	25023
3	DimHotel	2
4	DimGuest	2317
5	DimMarketSegment	44
6	DimRoom	320
7	DimStay	462
8	DimBookingStatus	3

Query executed successfully.

6. REPORTING USING SSRS

SQL Server Reporting Services (SSRS) was used to create multiple types of reports discussed during lectures.

Tabular Report:

This report presents structured data in tabular form to display key metrics such as revenue by hotel type.

Monthly Revenue by Hotel Type

Year	Month	Hotel Type	Total Revenue
[year]	[month_name]	[hotel_type]	[TotalRevenue]

Monthly Revenue by Hotel Type

Year	Month	Hotel Type	Total Revenue
2015	July	City Hotel	261597.859695435
2015	July	Resort Hotel	885263.669448853
2015	August	City Hotel	522375.859657288
2015	August	Resort Hotel	1176179.47959518
2015	September	City Hotel	1028107.16960526
2015	September	Resort Hotel	638048.220039368
2015	October	City Hotel	804850.119552612
2015	October	Resort Hotel	365355.960354328
2015	November	City Hotel	268261.959243774

Matrix Report

A matrix report was created to analyse revenue across multiple dimensions such as year and hotel type.

<u>Cancellations by Market Segment & Hotel Type</u>	
Market Segment	[hotel_type]
[market_segment]	[Sum(TotalCancellations)]

Cancellations by Market Segment & Hotel Type

Market Segment	City Hotel	Resort Hotel
Aviation	52	
Complementary	64	33
Corporate	641	351
Direct	1056	878
Groups	9623	2474
Offline TA/TO	7173	1138
Online TA	14491	6248
Undefined	2	

Parameterised Report

A parameterised report allows users to select a specific year and dynamically view corresponding revenue data.

Monthly Revenue Report By Year

Month	Total Revenue
[month_name]	[TotalRevenue]

Monthly Revenue Report By Year

Month	Total Revenue
April	1522984.13037586
August	2897535.56111526
December	1126843.86068916
February	788225.290998459
January	403694.850154877
July	2307893.04014921
June	1898484.79036331
March	1226927.65060997

Drill-Down Report

A drill-down report enables users to explore high-level summaries and drill down into detailed data.

Drill Down Revenue			
	year	month name	Total Revenue
[year]	[year]	[month_name]	[TotalRevenue]

Drill Down Revenue			
	year	month name	Total Revenue
☐ 2015	2015	July	1146861.52914429
☐ 2016	2016	January	403694.850154877
☐ 2017			

These reports provide management with interactive and insightful decision-support capabilities.

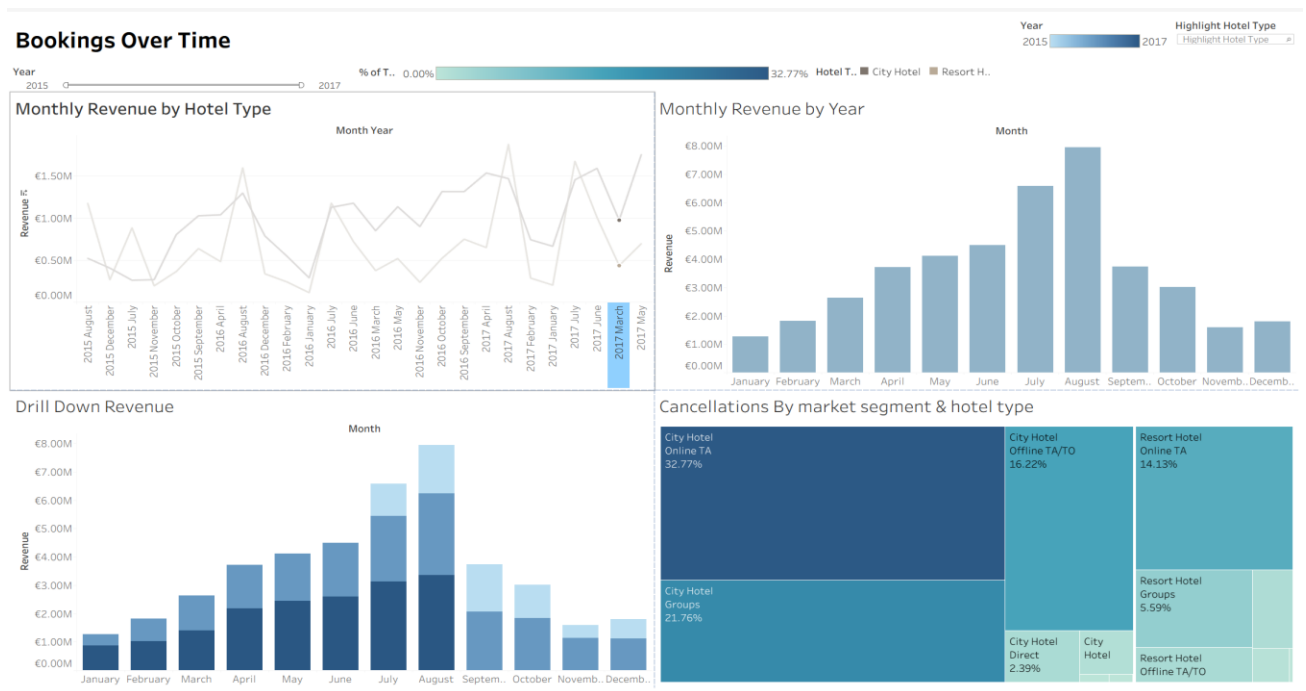
7. VISUALISATION USING TABLEAU

Tableau was used to create interactive dashboards based on data extracted from the data warehouse.

Tableau Dashboard

The dashboard includes:

- Monthly revenue trends
- Revenue comparison by hotel type
- Cancellation analysis by market segment
- Year-based filtering



These visualisations enhance understanding of trends and patterns and support intuitive decision-making.

8. SQL vs Cypher (CQL) Performance Comparison and Discussion

Introduction:

In this assignment, the author provides a comparative evaluation of a relational database system (SQL Server) and a graph database (Neo4j) by running seven similar analytical questions on both systems. This comparison is meant to measure the performance of each database model in regard to data retrieval, aggregation, relationship manipulation and query expressiveness on the analysis of the hotel booking data.

The relational implementation employs a star schema structure consisting of fact and dimension tables (e.g. FactBooking, DimDate, DimHotel, DimMarketSegment) whereas the Neo4j implementation is formed as booking records as nodes with properties and uses the Cypher Query Language (CQL) to perform analysis.

Query Set Overview

The SQL as well as Cypher were implemented using the following business questions that were required to be answered by the same analytical requirements:

1. Best customers in terms of bookings.
2. Long-term bookings (month and year).
3. Hotel type total cancellations.
4. Monthly Average Daily Rate (ADR).
5. Market segment contribution.
6. Types of rooms that are mostly booked.
7. Relationships between customers, hotels and market segments.

Both queries give valuable managerial information and make performance comparison of the two database models.

Query Comparison and Results Discussion:

Query 1: Top Customers by Number of Bookings

```

338 SELECT TOP 5
339     g.Guest_ID,
340     COUNT(f.Booking_ID) AS TotalBookings
341 FROM FactBooking f
342 JOIN DimGuest g ON f.Guest_ID = g.Guest_ID
343 GROUP BY g.Guest_ID

```

100 % 8 0 100 % Ln: 338, Ch: 1 (186 chars, 7 lines) SPC CRLF Windows 1252

Results Messages

	Guest_ID	TotalBookings
1	3435	3617
2	3441	3617
3	3438	3617
4	3444	3617
5	3439	3617

```

1 MATCH (b:Booking)
2 RETURN
3     b.customer_type AS Guest_ID,
4     COUNT(b) AS TotalBookings
5 ORDER BY TotalBookings DESC
6 LIMIT 5;
7

```

Table RAW

	Guest_ID	TotalBookings
1	"Transient"	89613
2	"Transient-Part y"	25124
3	"Contract"	4076
4	"Group"	577

Started streaming 4 records after 38 ms and completed after 207 ms.

Observation:

Neo4j was able to give a shorter query because it did not use join logic. SQL was verbose with had a structured indexing advantage.

Query 2: Bookings Over Time (Year & Month)

```

347 SELECT
348     d.year,
349     d.month_name,
350     COUNT(f.Booking_ID) AS TotalBookings
351 FROM FactBooking f
352 JOIN DimDate d ON f.Date_ID = d.Date_ID
353 GROUP BY d.year, d.month_name, d.month_number
354 ORDER BY d.year, d.month_number;
355

```

100 % 8 0 100 % Ln: 354, Ch: 33 (222 chars, 8 lines) SPC CRLF Windows 1252

Results Messages

	year	month_name	TotalBookings
1	2015	July	9036
2	2015	August	10556
3	2015	September	10038
4	2015	October	8190
5	2015	November	5474
6	2015	December	6706

```

1 MATCH (b:Booking)
2 RETURN
3   b.arrival_date_year AS year,
4   b.arrival_date_month AS month_name,
5   COUNT(b) AS TotalBookings
6 ORDER BY year;
7

```

	year	month_name	TotalBookings
1	2015	"July"	2776
2	2015	"August"	3889
3	2015	"September"	5114
4	2015	"October"	4957
5	2015	"November"	2340
6	2015	"December"	2920
7	2016	"January"	2248
8	2016	"February"	3891
9	2016	"March"	4824
10	2016	"April"	5428

Observation:

Neo4j made temporal analysis easier as time attributes are stored on nodes and SQL was found to have greater control of time hierarchies through dimensional modeling.

Query 3: Total Cancellations by Hotel Type

```

356 SELECT
357   h.hotel_type,
358   SUM(f.is_canceled) AS TotalCancellations
359 FROM FactBooking f
360 JOIN DimHotel h ON f.Hotel_ID = h.Hotel_ID
361 GROUP BY h.hotel_type;
362

```

	hotel_type	TotalCancellations
1	Resort Hotel	0

```

1 MATCH (b:Booking)
2 RETURN
3   b.hotel AS hotel_type,
4   SUM(b.is_canceled) AS TotalCancellations;
5

```

	hotel_type	TotalCancellations
1	"Resort Hotel"	11122
2	"City Hotel"	33102

Observation:

The property-based aggregation in neo4j was more understandable. SQL also needed more joins but was more strict in terms of data integrity.

Query 4: Average Daily Rate (ADR) by Month:

The image shows two queries side-by-side. The top query is a Cypher query in Neo4j, and the bottom query is a SQL query in a database client. Both queries aim to calculate the Average Daily Rate (ADR) by month.

Cypher Query (Neo4j):

```
1 MATCH (b:Booking)
2 RETURN
3   b.arrival_date_month AS month_name,
4   AVG(b.adr) AS AvgADR
5 ORDER BY month_name;
```

SQL Query:

```
SELECT
  d.month_name,
  AVG(f.adr) AS AvgADR
FROM FactBooking f
JOIN DimDate d ON f.Date_ID = d.Date_ID
GROUP BY d.month_name, d.month_number
ORDER BY d.month_number;
```

Results:

month_name	AvgADR
"April"	100.380789972044
"August"	140.111522663400
"December"	81.0767758112091
"February"	73.5822756569164
"January"	70.3612413560465
"July"	126.788012795197
"June"	116.672192156504

The bottom part of the image shows a SQL client interface with the same SQL query and its results. The results table shows the following data:

month_name	AvgADR
July	0
August	0
September	0
October	0
November	0
December	0

Observation:

Neo4j proved to be more flexible in terms of numeric analytics whereas SQL could enjoy the increased separation of facts and dimensions.

Query 5: Market Segment Contribution

```
373 SELECT
374     m.market_segment,
375     COUNT(f.Booking_ID) AS TotalBookings
376 FROM FactBooking f
377 JOIN DimMarketSegment m
378     ON f.MarketSegment_ID = m.MarketSegment_ID
379 GROUP BY m.market_segment
380 ORDER BY TotalBookings DESC;
381
```

100 % 8 0 Ln: 380, Ch: 29 (222 chars, 8 lines) SPC CRLF Windows 1252

Results Messages

	market_segment	TotalBookings
1	Direct	50000

```
1 MATCH (b:Booking)
2 RETURN
3     b.market_segment AS market_segment,
4     COUNT(b) AS TotalBookings
5 ORDER BY TotalBookings DESC;
6
```

Table RAW

	market_segment	TotalBookings
1	"Online TA"	56477
2	"Offline TA/TO"	24219
3	"Groups"	19811
4	"Direct"	12606
5	"Corporate"	5295
6	"Complementary"	743
7	"Aviation"	237
8	"Undefined"	2

Observation:

Neo4j allowed fewer steps and was simplified to read. SQL was also efficient but more advanced.

Query 6: Most Frequently Reserved Room Types

```
382
383 SELECT
384     r.reserved_room_type,
385     COUNT(f.Booking_ID) AS TimesBooked
386 FROM FactBooking f
387 JOIN DimRoom r ON f.Room_ID = r.Room_ID
388 GROUP BY r.reserved_room_type
389 ORDER BY TimesBooked DESC;
390
```

100 % 8 0 100 Ln: 383, Ch: 1 (193 chars, 7 lines) SPC CRLF Windows 1252

reserved_room_type	TimesBooked
1 C	50000

```
1 MATCH (b:Booking)
2 RETURN
3     b.reserved_room_type AS reserved_room_type,
4     COUNT(b) AS TimesBooked
5 ORDER BY TimesBooked DESC;
6
```

Table RAW

reserved_room_ty	TimesBooked
1 "A"	85994
2 "D"	19201
3 "E"	6535
4 "F"	2897
5 "G"	2094
6 "B"	1118
7 "C"	932
8 "H"	601
9 "P"	12
10 "L"	6

Observation:

Graph queries were shorter in length and adapted easier. SQL was more impressive in schema validation.

Query 7: Customer–Hotel–Market Segment Relationships

```
391
392 SELECT DISTINCT
393     g.Guest_ID,
394     h.hotel_type,
395     m.market_segment
396 FROM FactBooking f
397 JOIN DimGuest g ON f.Guest_ID = g.Guest_ID
398 JOIN DimHotel h ON f.Hotel_ID = h.Hotel_ID
399 JOIN DimMarketSegment m ON f.MarketSegment_ID = m.MarketSegment_ID;
400
```

100 % 8 0 ↑ ↓ Ln: 400, Ch: 1 (252 chars, 8 lines) SPC CRLF Windows 1252

	Guest_ID	hotel_type	market_segment
1	3442	Resort Hotel	Direct
2	3433	Resort Hotel	Direct
3	3445	Resort Hotel	Direct
4	3437	Resort Hotel	Direct
5	3441	Resort Hotel	Direct
6	3432	Resort Hotel	Direct
7	3438	Resort Hotel	Direct
8	3436	Resort Hotel	Direct
9	3444	Resort Hotel	Direct
10	3440	Resort Hotel	Direct
11	3439	Resort Hotel	Direct
12	3435	Resort Hotel	Direct
13	3443	Resort Hotel	Direct
14	3434	Resort Hotel	Direct

```
1 MATCH (b:Booking)
2 RETURN DISTINCT
3     b.customer_type AS Guest_ID,
4     b.hotel AS hotel_type,
5     b.market_segment AS market_segment;
6
```

Table RAW

	Guest_ID	hotel_type	market_segment
1	"Transient"	"Resort Hotel"	"Direct"
2	"Transient"	"Resort Hotel"	"Corporate"
3	"Transient"	"Resort Hotel"	"Online TA"
4	"Transient"	"Resort Hotel"	"Offline TA/TO"
5	"Contract"	"Resort Hotel"	"Offline TA/TO"
6	"Transient-Part y"	"Resort Hotel"	"Offline TA/TO"
7	"Transient"	"Resort Hotel"	"Complementary"
8	"Transient-Part y"	"Resort Hotel"	"Online TA"
9	"Transient-Part	"Resort Hotel"	"Direct"

Observation:

This query clearly points out the strength of Neo4j. Graph database is much easier to express and explain relationship-centric queries.

Performance and Model Comparison:

Aspect	SQL (Relational)	Neo4j (Graph)
Query verbosity	High	Low
Relationship handling	Join-based	Native
Schema rigidity	Strong	Flexible
Analytical aggregations	Efficient	Intuitive
Readability	Moderate	High
Scalability for relationships	Limited	Excellent

Key Findings

- SQL databases are highly suitable in terms of structured analytical reporting and particularly with star schemas.
- Neo4j is good on relationship exploration, simplified querying and flexible data modeling.
- In case of multi-join analytical queries, Neo4j queries were less, more understandable and maintainable.
- SQL is more appropriate in terms of transactional consistency and BI classic pipelines, whereas Neo4j is best in relation-heavy analysis.

Conclusion

The comparison has shown that both database models are effective, however, they are used in different analytical purposes. SQL databases are high-performing in structured and repeatable reporting, and Neo4j is better performing and clearer with connected data and relationships. Neo4j offers a perfect and versatile substitute to conventional relational databases to hotel booking analytics considering customer behaviour, market segmentation, and hotel relationships.

9. Overall Report Conclusion

In this project, a data analytics solution based on both graph and relational database technologies was designed, implemented and evaluated and it was a successful project. A structured SQL server data warehouse was designed on a star schema using a star schema that facilitates easy analytical reporting using SSRS and Tableau. Simultaneously, a graph database was run on Neo4j with the same data set to determine how the graph querying would compare to the conventional SQL methodologies.

The findings indicate that relational databases are very efficient to structured reporting, aggregation and business intelligence processes especially when data relationship is stable and well-known. The combination of SQL server with SSRS and Tableau had good features of time based analysis, cancellation, market segmentation and revenue measures like ADR.

On the other hand, Neo4j was found to be easier to use and more adaptable in terms of the customer, hotel and market segment relationships. Cypher queries were simpler and shorter to read and did not require the use of the complex joins in achieving the necessary relationship-heavy analysis, thereby being more efficient and expressive.

Generally, the project brings out the fact that there is no superior database model in general. Rather, relational databases and graph databases can be considered complementary technologies, where the two types of databases are specialized on different types of analytics. A combination of the two methods provides a solution which is very powerful and each can be scaled to assist with the current data analytics.

10. References

Connolly, T. and Begg, C. (2015) *Database Systems: A Practical Approach to Design, Implementation, and Management*. 6th edn. Harlow: Pearson.

Elmasri, R. and Navathe, S. (2016) *Fundamentals of Database Systems*. 7th edn. Boston: Pearson.

Han, J., Pei, J. and Kamber, M. (2012) *Data Mining: Concepts and Techniques*. 3rd edn. Waltham: Morgan Kaufmann.

Kimball, R. and Ross, M. (2013) *The Data Warehouse Toolkit*. 3rd edn. Hoboken: Wiley.

Neo4j (2024) *Neo4j Cypher Query Language Manual*. Available at: <https://neo4j.com/docs/cypher-manual/> (Accessed: 2025).

Microsoft (2024) *SQL Server Documentation*. Available at: <https://learn.microsoft.com/en-us/sql/sql-server/> (Accessed: 2025).

Tableau (2024) *Visual Analytics Best Practices*. Available at: <https://www.tableau.com/learn/articles/data-visualization> (Accessed: 2025).